ZeroRun: Reproducible test-result reuse for AI coding tools

Jishan Kapoor*

Independent researcher, Toronto, Canada

Abstract

AI coding tools repeatedly invoke tests, but command repetition alone does not establish reusable evidence. ZeroRun is open-source research software for returning an explicitly identified previous success of a deterministic, result-only test command. It combines declared input and runtime identities, authenticated results, isolated fresh execution, and command-line and coding-tool interfaces. A read-only hit path avoids unnecessary source staging. We provide source-bound experiments, an independent input-inventory oracle, and a descriptive audit of public OpenHands trajectories. The four-target study, completed through an explicit post-crash recovery, reconciles 168 fresh comparisons and 96 reused successes. The package enables reproducible investigation of validation overhead and reuse boundaries; it does not establish autonomous-agent acceleration or reproduce cached test output.

Keywords: AI coding tools, test-result reuse, research software, reproducibility, software testing

*Corresponding author.
Email address: kapoorjishan2@gmail.com (Jishan Kapoor)

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	ZeroRun 0.5.1; research release 2026-09-06
C2	Permanent GitHub link to code/repository used for this code version	`floxy-21/zerorun-research`; linked immutable source/evidence snapshot
C3	Legal Code License	MIT (ZeroRun code); third-party licenses retained; trajectory data CC-BY-4.0
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python; pytest; Docker; CLI; Model Context Protocol (MCP); Codex skill
C6	Compilation requirements, operating environments & dependencies	Python $\geq$3.10; no core Python dependencies. Whole-task reuse: Linux/amd64 and Docker. Experiments: digest-pinned Python 3.12 container and frozen dependencies.
C7	If available Link to developer documentation/manual	Version-pinned README: installation, interfaces, reproduction, and limitations
C8	Support email for questions	kapoorjishan2@gmail.com

Table 1: Code metadata. The public release preserves the tested runtime source and provides a separate, documented research harness.

1. Motivation and significance

Repository-level AI coding tools use test results when deciding whether an edit is satisfactory. SWE-bench and SWE-agent make this interaction a concrete research setting [1, 2]. A reused result must be distinguishable from a new execution: a previous success is not a fresh transcript, and the same shell command can observe different source, dependencies, or external state. ZeroRun addresses a restricted but useful problem: exposing the previous successful exit status of a deterministic test command under an unchanged declared execution identity. Its primary users are developers integrating testing tools and researchers measuring the trade-off between validation cost and avoided execution. A relevant integration is a coding tool that revisits an already tested source state and needs its test status, while requesting fresh execution whenever it needs a new transcript. ZeroRun does not learn a caching policy or predict patch correctness. Users configure an existing pytest target, inspect its eligibility, and invoke it through a CLI or repository-bound coding-tool interface.

The foundations are established. Regression-test selection avoids tests considered unaffected by a change [3, 4]; build caches such as Bazel store action results and outputs [5]. ToolCaching investigates caching LLM tool calls, while TVCache supports stateful tool reuse in agent post-training [6, 7]. The SoftwareX tool eidos provides an adjacent contribution in LLM function integration and validation [8]. Therefore, neither content hashing nor attaching a cache to an AI interface is claimed as a new algorithm.

The software contribution has three connected parts: a repository-bound interface that distinguishes reused status from fresh evidence; an implementation that avoids execution staging on eligible whole-file hits; and an executable evaluation kit that checks input boundaries and compares full request costs with fresh outcomes. Unlike feature/TTL-guided tool caching or trajectory-prefix reuse during agent post-training, the demonstrated mode revalidates a declared local test-task identity and returns status metadata without reconstructing arbitrary effects or transcripts. This is a difference in operating contract, not a claim of superiority. The accompanying evidence distinguishes three questions: did reused status agree with fresh execution, was reuse actually eligible, and did avoided execution exceed checking overhead? An integrator can therefore evaluate both whether reuse is allowed and whether it is worthwhile.

2. Software description

2.1. Architecture and execution contract

The Python package separates manifest validation, input fingerprinting, result storage, isolated execution, and interfaces. A version-2 whole-task manifest identifies a command, declared input paths, environment policy, and digest-pinned Linux/amd64 container. Its supported cache contract requires a result-only computation, no output artifacts, and no cached stdout or stderr. The input closure and determinism are operator assumptions, not facts inferred from passing tests.

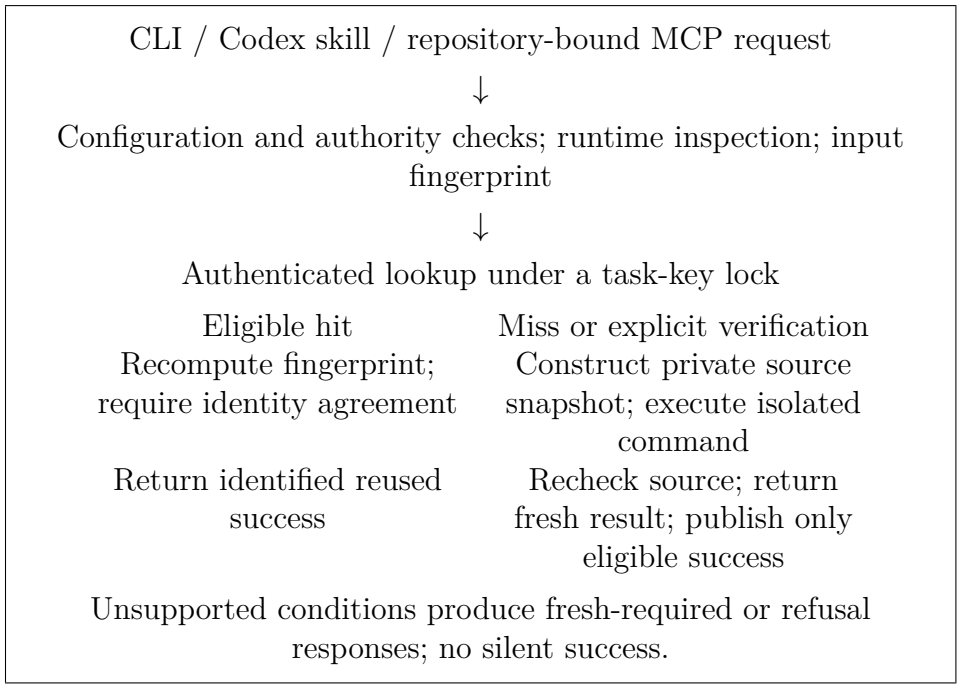

Figure 1: Whole-file result-reuse path. A hit returns result metadata, not the prior test transcript. The diagram describes software control flow, not an autonomous-agent evaluation.

The identity includes declared content, relevant command-source content, directory modes, command and environment policy, and inspected runtime identity. A matching cache filename is insufficient: the entry must authenticate and satisfy the configured contract. On a miss, ZeroRun stages a private source copy and runs it read-only in a resource-constrained, network-disabled container. A post-execution check detects source drift before publishing a success. Failed commands do not become reusable successes. Explicit verification can quarantine a stored result that conflicts with fresh execution.

The optimized whole-file hit path computes the full fingerprint, locks and authenticates the entry, then independently recomputes the full fingerprint. It returns a hit only if both identities and their payloads agree. No repository code executes and no execution copy is constructed on this path. Misses, forced execution, verification, and symbol-projection tasks retain the existing snapshot path. The optimization changes the order of work, not the cache eligibility threshold.

This is conditional correctness, not a soundness proof for arbitrary Python. An undeclared dependency, nondeterministic test, or violated host assumption can invalidate reuse. Two filesystem scans are not an atomic snapshot against arbitrary concurrent host mutations. Unsupported or insufficiently reviewed work must execute fresh or remain refused.

2.2. Interfaces and functionality

The CLI exposes structured results identifying reused success, fresh success, fresh failure, uncertainty, and refusal. The Codex skill and repository-bound MCP server expose the same distinction. Normal agent-facing reuse additionally requires exact-configuration authorization outside repository-controlled files; generating observation candidates does not authorize reuse. Laboratory fixtures used by the experiments authenticate cache records but are not production operator reviews, and their private keys are excluded from the artifact.

The package also contains a fine-grained pytest path. Because omitted tests can affect fixtures, callbacks, shared state, or ordering, it conservatively requires fresh execution when qualification is incomplete. That path is included for reproducibility, but it is not the demonstrated acceleration mechanism in this article. Whole-task reuse does not require individual tests to be independent; it requires the complete requested command to satisfy the stated contract.

On a hit, stdout and stderr are not reconstructed. A coding agent requiring failure details, coverage, generated files, or a new transcript must request fresh execution. This explicit API choice prevents conflating successful status reuse with general stateful tool replay. Installation and integration checks are automated; no external user adoption is asserted.

3. Illustrative examples

3.1. Repeat, failure, and restoration

The public artifact provides an executable example using a fixed pytest target. Starting with an empty store, the first request executes. Two unchanged requests can reuse its success. Appending an always-failing test changes the

declared identity; both the first failing request and its repeat execute and fail. Restoring the original bytes permits the original success to be found again, followed by another reuse. The expected seven-response sequence is:

Request	Expected whole-task response
Seed	`MISS_EXECUTED`
Two unchanged repeats	`HIT_REUSED` each
Added failure; repeated failure	`MISS_FAILED` each
Restoration; restored repeat	`HIT_REUSED` each

The example records an independent fresh full-target check after every request, including hits. It is a constructed validation sequence, not an estimate of the repetition frequency in real development.
For a configured target named `tests`, the CLI form is:

```
zerorun --manifest .zerorun.json --json explain tests
zerorun --manifest .zerorun.json --json run tests
zerorun --manifest .zerorun.json --json run tests --verify
```

The first command inspects eligibility; the second requests execution or eligible reuse; the third checks a stored result by executing fresh. The named target must already exist in the reviewed manifest. These commands do not create an input-completeness review or override missing authority. The example harness supplies its own explicitly laboratory-only configuration and retains the complete returned JSON, including `status` and `exit_code`. In an integration, `HIT_REUSED` is labeled as a previous success, a fresh failure follows the ordinary diagnostic path, and a refusal must not be converted into success. If a new transcript is required, the caller executes fresh. This small decision boundary is shared by the CLI and coding-tool interfaces; it is not an assumed property of an autonomous agent.

3.2. Real AI-workflow observations

We also provide a read-only analysis of the public SWE-rebench OpenHands trajectories [9, 10]. After inspecting ten separate pilot rows, a fixed SHA-256 ordering selected 128 rows from the remaining 67,064. Selection did not use repository, resolution, repetition, or performance. The raw responses, source notices, retrieval receipts, exclusions, and deterministic parser are available. An initial rate-limited attempt is retained, followed by a complete recovery of the same selected rows.
Of the 128 selected episodes, 122 had the required action/observation pairing; six were excluded for unmatched execution records, not replaced. The valid set spans 104 repositories and 120 issues. It contains 745 supported

direct pytest invocations and 120 exact-command repeat pairs. All repeats have intervening barriers: 112 contain a reported editor mutation and eight contain other unmeasured effects. These are observations about recorded actions, not 120 cache hits. The parser excludes compound-shell commands from the direct-command category and preserves clipped outputs and missing exit markers.

This example makes a practical distinction visible: repeated command text is an inadequate cache key for source-editing agents. Even an apparent restoration requires independent source and environment reconstruction. A purposively selected django-environ episode provides a positive, bounded mechanism example. We reconstructed its recorded source edits, added a fresh check at the intermediate edited state, and ran four controlled requests (Table 2). After the inverse edit, the complete declared source/runtime identity and original target's cache key rejoined; the restored request returned one explicitly identified reused success, agreeing with an independent fresh 14-test outcome. The intervening collection command selected no tests and exited 5; it remained a fresh failure, not a reusable success.

Controlled request	Actual response	Exit	Fresh nodes
Seed target	`MISS_EXECUTED`	0	14
Edited target (extra check)	`MISS_EXECUTED`	0	15
Restored collection	`MISS_FAILED`	5	0
Restored target	`HIT_REUSED`	0	14

Table 2: One source-restoration case. The 15-test request is additional counterfactual instrumentation, not a test call made by the recorded agent. Every row includes a separate fresh full-target oracle.

This standalone case ran after the timing VM interruption, with independently checked source bindings. Retained failed attempts exposed a missing Git-fixture marker; a separately versioned producer initialized local metadata while preserving the baseline's masking guard. The successful run uses Python 3.12.14 and pytest 9.1.1, not the recorded Python 3.9 environment. It demonstrates this content-restoration mechanism, not historical output replay, unchanged autonomous-agent behavior, or population-wide benefit.

4. Impact

4.1. Reproducible validation and measured cost

The software supports experiments on how much work validation itself consumes, when conservative qualification prevents reuse, and how cache admission differs from command repetition. Researchers can replace a target or

change an input boundary while preserving the fresh-oracle comparison and explicit denominators. Raw invocation records, source hashes, fixed manifests, failure states, setup receipts, and analysis scripts make unfavorable results inspectable rather than dependent on summary claims.

The evaluation kit supports a concrete adoption decision: first qualify the result-only contract, then check agreement and failure behavior, and finally compare the full request sequence with direct execution, including available setup cost. Fast individual hits are insufficient if checking and miss costs erase their benefit. The supplied ablation isolates the staging change; it is not a head-to-head evaluation against another caching system.

The original comparison used five pinned Python libraries: packaging, pycparser, Colorama, Click, and more-itertools. Ordinary pytest is the uncached reference; disabling only the optimized hit path gives a snapshot-first ablation, not an independent competing cache. Two reversed-order seven-request sequences per library yielded 70 fresh-result agreements and 40 optimized-arm whole-task hits. The original mean conditional hit latency reduction against the ablation was 67.3–71.0%. Complete-sequence direct/optimized ratios were 0.85–2.09. For more-itertools, optimized execution was 16.6% slower overall than snapshot-first reuse, despite faster hits. These results are retained, including large lifecycle delays and their order sensitivity.

To examine timing stability, a prospectively recorded extension used six independently initialized blocks on each of the four previously observed short subjects, covering every ordering of direct pytest, snapshot-first reuse, and optimized reuse. The seven request states were unchanged. All arms used the same 900-second test-execution bound and container resource limits. Docker control phases in the direct/oracle helpers used 30-second limits, whereas product inspection and cleanup used 60-second limits; complete invocation timings include these different lifecycle paths. Timings cover complete helper/API invocations; host telemetry and receipt writing are outside every timed boundary. Dependency preparation and block setup are separately retained. This is a replication on seen subjects, not a new holdout or a replacement for more-itertools.

Across 24 completed planned blocks, all 168 requests agreed with their fresh full-target checks and showed the expected cache behavior, including 96 optimized hits and 72 non-hit requests. Table 3 reports full-sequence and setup-inclusive ratios, all-block spread, and conditional median-hit savings separately. No completed block or outlying invocation within those blocks is dropped; interrupted-attempt costs are reported separately. The complete-block record contains 504 timed arm invocations and 168 separate fresh-oracle captures. The original five-subject results are not pooled with this extension. A VirtualBox host assertion aborted the VM after 21 complete blocks and

part of Packaging block four. A separately recorded amendment reran only the unfinished preselected blocks four through six from independent initial states, using the unchanged frozen driver. The original protocol, partial invocation files, zero-byte crash remnants, failed recovery preflight, and surviving timings remain available. This is recovered coverage of the plan, not uninterrupted completion of its original no-retry protocol. Packaging's unflushed original environment-setup total is unavailable; its setup-inclusive ratio is therefore not reported. The interrupted attempt adds 1 complete request, 1 partially recorded request, and an unflushed invocation-only directory with no outcomes. Its surviving arm costs are retained separately; including those costs changes Packaging's direct/optimized ratio to 1.029.

Subject	D/F	D/F + setup	Block range	Hit saving (%)
click	1.05	1.05	0.89–1.64	68.1
pycparser	0.89	0.91	0.78–1.04	68.8
colorama	1.01	1.01	0.98–1.11	67.6
packaging	1.04	—	0.88–1.32	70.8

Table 3: Complete replicated sequence cost versus conditional hit benefit. D/F is summed direct time divided by summed optimized time; above one favors reuse. Setup-inclusive ratios allocate common dependency preparation and each block's materialization to both arms. Block range retains all six complete sequences. Hit saving compares median optimized and snapshot-first hit latency, not end-to-end agent time.

An independent standard-library input-inventory oracle checks boundary changes without importing ZeroRun's fingerprint implementation. Linux validation covers 32 production/oracle comparisons and six explicit oracle-scope refusals. Cases include same-size content changes with restored modification time, helper/configuration/data edits, additions, deletion, renaming, directory modes, restoration, command and environment changes, missing inputs, and symlinks. A new *undeclared* top-level file changes neither declared identity: the test documents this limitation rather than claiming automatic closure discovery. These checks establish implementation behavior within the tested boundary, not universal determinism.

The source-bound engineering record includes successful Python 3.10–3.14 CI regression runs and a Windows run with 1,593 passing tests, 55 skips, and 19 passing subtests. Selected public-package tests separately passed 465 tests with 21 skips and eight passing subtests; this is not a second run of the entire development suite. A separate 100-repository CLI/skill/MCP integration exercise returned 99 passes and one safe refusal, not 100 upstream test-suite executions or autonomous AI tasks. A five-library revision evaluation recorded 200 fresh comparisons but no accepted fine-grained node reuse;

the pre-existing product performance gates remain unmet. These distinct evaluations are not pooled into a single reliability percentage.

4.2. Scope, reuse, and limitations

For an integrator, the benefit is a small Python package with a result contract and review boundary that can be inspected before enabling reuse. For a researcher, the benefit is an executable evaluation kit linking an interface response to exact implementation bytes, fresh outcomes, and full invocation cost. The public MIT release permits adaptation and commercial reuse of ZeroRun-owned code; third-party data and software keep their own licenses. The evidence does not establish the prevalence of reusable tasks, population-wide speedup, model-token savings, improved patch quality, user productivity, or commercial demand. The trajectory audit is descriptive and resource-bounded; repository and issue clustering is reported. Its zero barrier-free pairs do not prove zero state-restoration opportunities, but they prevent interpreting repetition as measured eligibility. The controlled library targets are convenience workloads on one constrained VM, and original and replicated observations are not statistically independent datasets. The VM shared a Windows host with publication and validation tooling; host activity outside the VM was not fully isolated. Complete-block spreads and influential observations are reported without treating thousands of nested test nodes as independent samples.

No external users, production deployments, or third-party publications using ZeroRun are claimed. Potential use in AI-assisted software development motivates the software, but adoption and economic benefit remain future evaluations. The next application-level evaluation would measure how coding agents interpret reused-status responses and whether suitable tasks occur often enough to improve complete workflows. The present artifact supplies the execution and measurement components for that study, not its outcome.

5. Conclusions

ZeroRun provides reusable software for explicitly identified test-status reuse in AI coding-tool integrations. Its whole-file hit path removes execution staging when no code will run, while retaining full identity checks and the existing fresh path. The public package combines runnable examples, fresh-oracle comparisons, independent boundary checks, and a real-trajectory observation tool. The results support conditional utility and expose overhead and applicability limits. The release is a research and integration artifact, not evidence of universal AI acceleration.

Data and software availability

Code, raw evidence, analysis scripts, and reproduction instructions are provided at `https://github.com/floxy-21/zerorun-research`. Table 1 pins the release used by the article; the source-binding manifest connects it to runtime revision `86f42289c2f59a74b1f642f0b20d1a26b3d55e57`. Public trajectory responses are redistributed with CC-BY-4.0 notices and unmodified source attribution. Private cache-authentication fixture keys and operator authority are not distributed.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interests

Jishan Kapoor develops ZeroRun and has an interest in its potential commercialization. No external funding or established commercial adoption is reported.

Declaration of generative AI and AI-assisted technologies

OpenAI Codex was used extensively to assist software and experiment implementation, analysis, literature checking, manuscript drafting, and document preparation. It is not an author. The sole human author is responsible for reviewing the evidence, verifying references and claims, and approving the submitted version.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, SWE-bench: Can language models resolve real-world GitHub issues?, in: The Twelfth International Conference on Learning Representations, 2024.
URL `https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html`

[2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, SWE-agent: Agent-computer interfaces enable automated software engineering, in: Advances in Neural Information Processing Systems 37, 2024. `doi:10.52202/079017-1601`.

[3] G. Rothermel, M. J. Harrold, A safe, efficient regression test selection technique, ACM Transactions on Software Engineering and Methodology 6 (2) (1997) 173–210. `doi:10.1145/248233.248262`.

[4] S. Yoo, M. Harman, Regression testing minimization, selection and prioritization: A survey, Software Testing, Verification and Reliability 22 (2) (2012) 67–120. doi:10.1002/stvr.430.

[5] Bazel Project, Remote caching, official documentation; accessed 2026-09-05 (2026).
URL https://bazel.build/remote/caching

[6] Y. Zhai, D. Shen, J. Luo, B. Yang, ToolCaching: Towards efficient caching for LLM tool-calling, preprint, version 1 (2026). arXiv:2601.15335, doi:10.48550/arXiv.2601.15335.
URL https://arxiv.org/abs/2601.15335

[7] A. V. Kumar, B. Kataria, B. Oh, E. Manzoor, R. Singh, TVCache: A stateful tool-value cache for post-training LLM agents, preprint, version 1 (2026). arXiv:2602.10986, doi:10.48550/arXiv.2602.10986.
URL https://arxiv.org/abs/2602.10986

[8] J. F. Aldana-Martín, A. Benítez-Hidalgo, J. F. Aldana-Montes, eidos: A modular approach to external function integration in LLMs, SoftwareX 31 (2025) 102290. doi:10.1016/j.softx.2025.102290.
URL https://doi.org/10.1016/j.softx.2025.102290

[9] M. Trofimova, A. Shevtsov, I. Badertdinov, K. Pyaev, S. Karasik, A. Golubev, OpenHands trajectories with Qwen3-Coder-480B-A35B-Instruct, nebius dataset and accompanying blog, CC-BY-4.0; authors and year from the archived dataset citation. Observed revision 35455389ab51bf5e2306bfd436ef72d0f98bf882; accessed 6 September 2026. Exact retrieved responses archived with the article (2025).
URL https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories

[10] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, B. Yangel, SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, in: Advances in Neural Information Processing Systems 38, 2025. doi:10.52202/085713-0788.
URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/21bec6ace947b1b58967b945c8ac0f10-Abstract-Datasets_and_Benchmarks_Track.html

Current executable software version

ZeroRun 0.5.1, Python command-line package. The public release includes installation instructions and automated checks; Linux/amd64 Docker execution is required for the demonstrated whole-task reuse mode.